

Database Management System

Course Code: CS210

Dr. Gunjan Chugh

Assistant Professor

**Dept. of Computer Science Engineering
Delhi Technological University**

Course Description

Unit No.	Description of Unit
Unit 1	Introduction: Database system concepts and its architecture, data models, schema and instances, data independence and database language and interface, data definition languages, DML, overall database structure. Data Modeling using Entity Relationship Model: E-R model concept, notation for ER diagrams, mapping constraints, keys, concept of super key, candidate key, primary key, generalizations, aggregation, reducing ER diagrams to tables, extended ER model.
Unit 2	Relational Data Model and Language: Relational data model concepts, integrity constraints, keys, domain constraints, referential integrity, assertions, triggers, foreign key. Relational algebra, relational calculus, domain and tuple calculus, SQL data definition queries and updates in SQL.

Course Description

Unit No.	Description of Unit
Unit 3	Database Design: Functional dependencies, normal forms – 1NF, 2NF, 3NF and BCNF, multi-valued dependencies, fourth normal form, join dependencies and fifth normal form. Inclusion dependencies, lossless join decompositions, normalization using FD, MVD and JD's, alternative approaches to database design.
Unit 4	File Organization, Indexing and Hashing: Overview of file organization techniques, indexing and hashing – basic concepts, static hashing, dynamic hashing, ordered indices, multi-level indexes, B-tree index files, B+ tree index files, buffer management. Transaction Management: Transaction concept, schedule and recoverability, serializability of schedules, conflict and view serializable schedule, testing of serializability.
Unit 5	Concurrency Control: Lock-based protocols for concurrency control, timestamp-based protocols for concurrency control, concurrency control in distributed systems, multiple granularities and multi-version schemes, deadlock handling. Recovery System: Failure classification, log-based recovery, recovery with concurrent transactions, buffer management. Case Studies: Commercial databases – Oracle, Postgres, MySQL, NoSQL.

Recommended Reading Material

S. No.	Name of Books / Authors / Publishers
1	Data Base Concepts by Korth, Silbertz, Sudarshan, McGraw-Hill
2	Fundamentals of Database Systems by Elmasri, Navathe, Pearson
3	An Introduction to Database Systems by C. J. Date, Kannan, Pearson

Course Outcomes(CO)

Course Outcomes (CO)	
CO1	Provide an overview of database system fundamentals, architecture, data models, schemas, and ER diagram concepts.
CO2	Demonstrate the use of relational data models, integrity constraints, relational algebra/calculus, and SQL.
CO3	Develop database schemas using normalization techniques for efficient and consistent data organization.
CO4	Address file organization methods, indexing, hashing, and transaction processing in database systems.
CO5	Discuss concurrency control and recovery mechanisms in commercial and NoSQL database systems.

Evaluation Criteria

Components	Maximum Marks
CWS	15 (Attendance, Assignments/Quiz, Class Performance)
PRS	25 (Practical)
MTE	20 (Mid Term)
ETE	40 (End Term)
Total	100

Google Classroom

- **The study material, Assignment, Quiz, and all other instructions related to the Course will be posted on Google Classroom.**
- All students are requested to join the **DBMS, Section-A7** Google Classroom with the following **Code**:

64qooth2

Contents of Unit 1(Part 1):

INTRODUCTION

- Database system concepts
- Architecture
- Schema and instances
- Data models
- Data independence
- Data Definition Language(DDL)
- Data Manipulation Language(DML)
- Overall database structure

What is a Database?

- A **database** is an organized collection of data that is stored and accessed electronically.
- The **primary purpose of a database is to store data in a structured way** so that it can be easily retrieved, managed, and updated.
- A database can hold information related to a **specific domain**—such as employee records, inventory data, customer information, etc.—and allows multiple users and applications to interact with the data concurrently.

Key Characteristics:

- Structured format using tables, rows, and columns
- Persistent storage
- Supports querying and reporting
- Ensures data consistency and integrity
- Databases can be **small (like a personal contact list)** or **large (like Amazon's product database)**.

Example:

- A **library database** may contain information about books, authors, borrowers, and transactions.
- A **bank database** may store customer details, account numbers, balances, and transaction history.

Problems with Traditional File-Based Systems

Before databases, data was stored in **file-based systems** which had several limitations:

- **Data Redundancy:** Duplication of data across multiple files
- **Inconsistency:** Conflicting data in different files
- **Difficulty in Accessing Data:** Requires extensive programming to retrieve specific information
- **Poor Data Security:** Limited or no control over access to sensitive information
- **Lack of Integrity:** Difficult to enforce data accuracy and constraints

Problem in File-Based System	Traditional File System	How DBMS Solves It
Data Redundancy	Same data is stored in multiple files, wasting space	Data is stored once and shared
Data Inconsistency	Different files may have different values for the same data	Ensures consistent data
Data Isolation	Data is scattered across many files and formats	Centralized data storage
Difficult Data Access	New queries require new programs	Easy queries using SQL
Lack of Data Integrity	No strict rules to maintain correct data	Integrity constraints enforced
Security Issues	Limited access control	Strong user authentication & authorization
Concurrency Problems	Multiple users can overwrite data	Controlled multi-user access
Poor Data Sharing	Files are application-specific	Data can be shared easily
Backup & Recovery Issues	Manual and unreliable	Automatic backup and recovery
Program–Data Dependence	File structure changes require program changes Dr. Gunjan Chugh, Assitant Professor, DTU	Data independence

Benefits of Using a Database

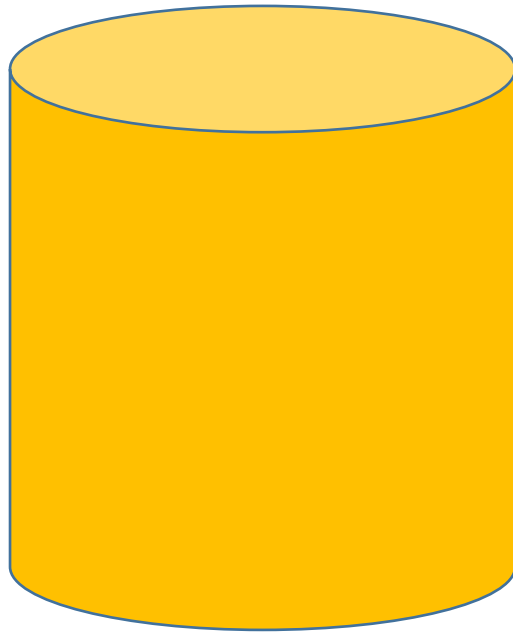
- Centralized data management
- Efficient data access and retrieval
- Reduced redundancy and improved consistency
- Better security and access control
- Scalability and concurrency for multiple users

Database System

A **database system** refers to the integrated framework that includes:

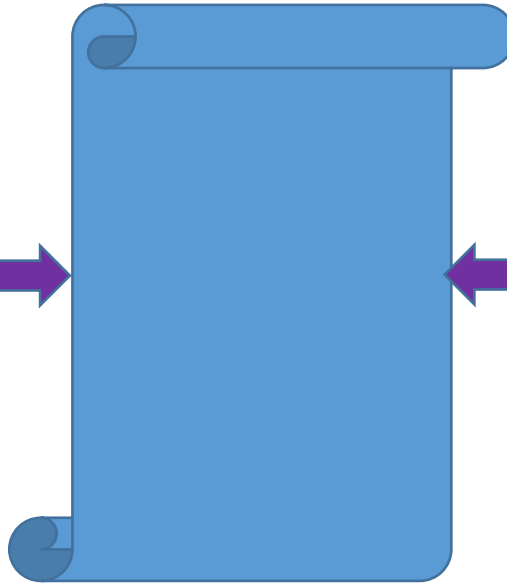
- The **database** itself (data storage)
 - The **Database Management System (DBMS)** software
 - The **users** who interact with the system
 - The **applications** that access the database through queries
-
- A **Database Management System (DBMS)** is a software application that interacts with end-users, applications, and the database to capture and analyze data.
 - It provides tools and interfaces to store, retrieve, update, and manage data efficiently.
 - A **DBMS** is software that **manages databases**.

DATABASE



Organized
collection of
related data

DBMS



Software which is used to
manage the database. e.g.
MS Access, MYSQL, Oracle

USER



User communicates
with DBMS interface
to fulfil requirement

Functions of a DBMS

- **Data Storage Management** – Stores data efficiently on disk.
- **Data Retrieval** – Allows users to query data using languages like SQL.
- **Data Security** – Controls who can access or modify data.
- **Data Integrity** – Ensures that data is accurate and consistent.
- **Backup and Recovery** – Restores data in case of failure.
- **Concurrent Access Control** – Allows multiple users to work simultaneously without conflicts.

Example of DBMS:

- **Oracle** – widely used in banking and enterprises.
- **MySQL** – popular in web applications.
- **MongoDB** – used for storing JSON-like documents.

Feature	MySQL	Oracle	MongoDB
Type	Relational DB (RDBMS)	Relational DB (RDBMS)	NoSQL DB (Document-oriented)
Data Structure	Tables (rows & columns)	Tables (rows & columns)	JSON-like documents (flexible schema)
Schema	Fixed / predefined	Fixed / predefined	Schema-less / dynamic
Query Language	SQL	SQL (PL/SQL)	MongoDB Query Language (MQL)
Cost	Free (Community), Paid (Enterprise)	Paid (Enterprise license)	Free (Community), Paid (Enterprise)
Best For	Learning SQL, labs, small-medium apps	Large enterprises, complex applications	Handling big/unstructured data, web apps, real-time apps
Ease of Learning	Easy	Harder, complex	Moderate, flexible
Use Case Example	Student lab exercises, small websites	Banks, airlines, ERP systems	Social media apps, real-time analytics, JSON-heavy apps

Core Functions of DBMS

- Data Definition:** Define the structure of data using DDL (Data Definition Language)
- Data Manipulation:** Insert, update, delete, and query data using DML (Data Manipulation Language)
- Data Security:** Manage access rights and user roles
- Data Integrity:** Enforce rules and constraints on data
- Transaction & Concurrency Management:** Ensure reliable and consistent transactions (ACID properties)

Core Functions of DBMS

Function	Purpose	What can be done?	Example
Data Definition	Define database structure	Creates tables, fields, data types, constraints	CREATE TABLE Student(StudentID INT PRIMARY KEY, Name VARCHAR(50));
Data Storage Management	Store data efficiently	Manages files, indexes, disk storage	Creating an index on StudentID for faster search
Data Manipulation	Modify and retrieve data	Inserts, updates, deletes, selects records	INSERT INTO Student VALUES (101,'Amit');
Query Processing	Execute queries efficiently	Optimizes and executes SQL queries	SELECT Name FROM Student WHERE StudentID=101;
Transaction Management	Maintain ACID properties	Ensures reliable transactions	Bank transfer using BEGIN, COMMIT, ROLLBACK
Concurrency Control	Support multiple users	Prevents conflicts in simultaneous access	Two users updating same account—DBMS uses locking

Core Functions of DBMS

Function	Purpose	Explanation	Example
Integrity Enforcement	Maintain data correctness	Enforces keys and constraints	Foreign key prevents invalid StudentID
Security & Authorization	Protect data	Controls user permissions	GRANT SELECT ON Student TO Faculty;
Backup & Recovery	Prevent data loss	Restores database after failure	Crash recovery using transaction logs
Data Independence	Separate data from programs	Data changes don't affect applications	Adding column Email without changing code
Metadata Management	Manage data dictionary	Stores schema and constraints	DBMS stores table and column definitions

Components of a Database System

- **Hardware:**

- Servers, storage devices, network devices.
- Example: Hard disks storing tables of a bank database.

- **Software (DBMS):**

- Manages the database and provides tools for query and reporting.
- Example: SQL Server, PostgreSQL.

- **Data:**

- The actual information stored.
- Example: Customer names, product prices, student grades.

- **Users:**

- **Database Administrator (DBA):** Manages and maintains the database.
- **End Users:** Use applications to interact with data.
- **Application Programmers:** Build software to manipulate data.

Advantages of DBMS

1. Data Redundancy Control

DBMS minimizes data duplication through centralized control, reducing storage costs and improving consistency.

2. Data Consistency

Data is updated in one place and reflected everywhere, ensuring consistent and accurate information across the system.

3. Improved Data Security

DBMS allows administrators to define user roles, authentication, and access rights to protect sensitive information.

4. Better Data Integration

Enables seamless integration of data across departments or domains, supporting better analysis and reporting.

Advantages of DBMS

5. Enhanced Data Sharing

Multiple users and applications can access the same database concurrently without conflicts.

6. Backup and Recovery

DBMS provides automated backup and recovery solutions to protect data from failures.

7. Data Integrity and Constraints

Ensures data validity using keys, constraints (e.g., NOT NULL, UNIQUE, CHECK), and referential integrity.

8. Transaction Support

Supports ACID (Atomicity, Consistency, Isolation, Durability) properties for reliable transactions.

9. Query Optimization

DBMS engines optimize queries to improve data retrieval speed and reduce load on resources.

DBA (Database Administrator)

- A **Database Administrator (DBA)** is a **specialized IT professional** responsible for the **overall management, maintenance, security, and performance of databases** in an organization.
- A DBA is responsible for database design, security, performance, backup, and overall database management.

Role of a DBA

1. Database Design & Setup

- Defines database schema
- Creates tables, indexes, and views
- Chooses appropriate data models and storage structures

2. Security & Authorization

- Manages user accounts and roles
- Grants and revokes access privileges
- Ensures data confidentiality and privacy

3. Performance Monitoring & Tuning

- Optimizes queries
- Manages indexes
- Improves response time and throughput

Role of a DBA

4. Backup & Recovery

- Takes regular backups
- Restores data after failure
- Implements disaster recovery plans

5. Data Integrity & Consistency

- Enforces constraints
- Maintains ACID properties
- Handles concurrency control issues

6. Maintenance & Monitoring

- Monitors database health
- Applies patches and upgrades
- Manages storage space

DBMS ARCHITECTURE

- **DBMS architecture describes how data is stored, accessed, and managed in a database system and how users interact with the database.**
- It defines:
 - **The levels of abstraction**
 - **The components involved**
 - **The flow of data between users and storage**

Types of DBMS Architecture

There are several types of DBMS architectures that we use according to the usage requirements.

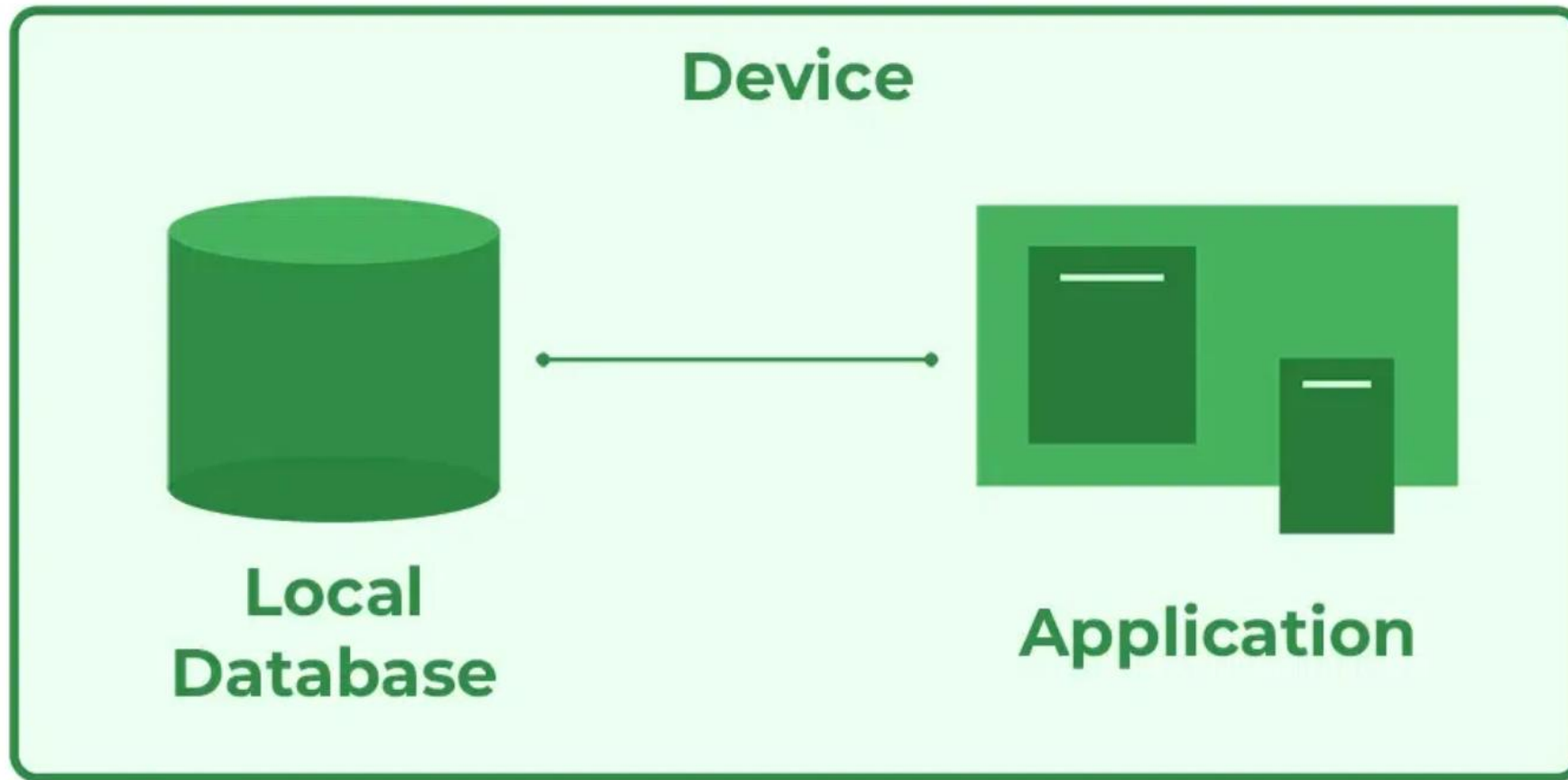
- 1-Tier Architecture
- 2-Tier Architecture
- 3-Tier Architecture

1-Tier Architecture

- In 1-Tier Architecture, the **user works directly with the database on the same system.**
- This means the **client, server and database are all in one application.**
- The user can open the application, interact with the data and perform tasks without needing a separate server or network connection.

Example

- A common example is **Microsoft Excel**. Everything from the user interface to the logic and data storage happens on the same device. The user enters data, performs calculations and saves files directly on their computer.
- **MS Access application** with a database stored on the same computer
- **MySQL installed locally** and queries executed using MySQL Workbench on the same system



1-tier architecture

Advantages of 1-Tier Architecture

- **Simple Architecture:** 1-Tier Architecture is the most simple architecture to set up, as only a single machine is required to maintain it.
- **Cost-Effective:** No additional hardware is required for implementing 1-Tier Architecture, which makes it cost-effective.
- **Easy to Implement:** 1-Tier Architecture can be easily deployed and hence it is mostly used in small projects.

Disadvantages of 1-Tier Architecture

- **Limited to Single User:** Only one person can use the application at a time. It's not designed for multiple users or teamwork.
- **Poor Security:** Since everything is on the same machine, if someone gets access to the system, they can access both the data and the application easily.
- **No Centralized Control:** Data is stored locally, so there's no central database. This makes it hard to manage or back up data across multiple devices.
- **Hard to Share Data:** Sharing data between users is difficult because everything is stored on one computer.

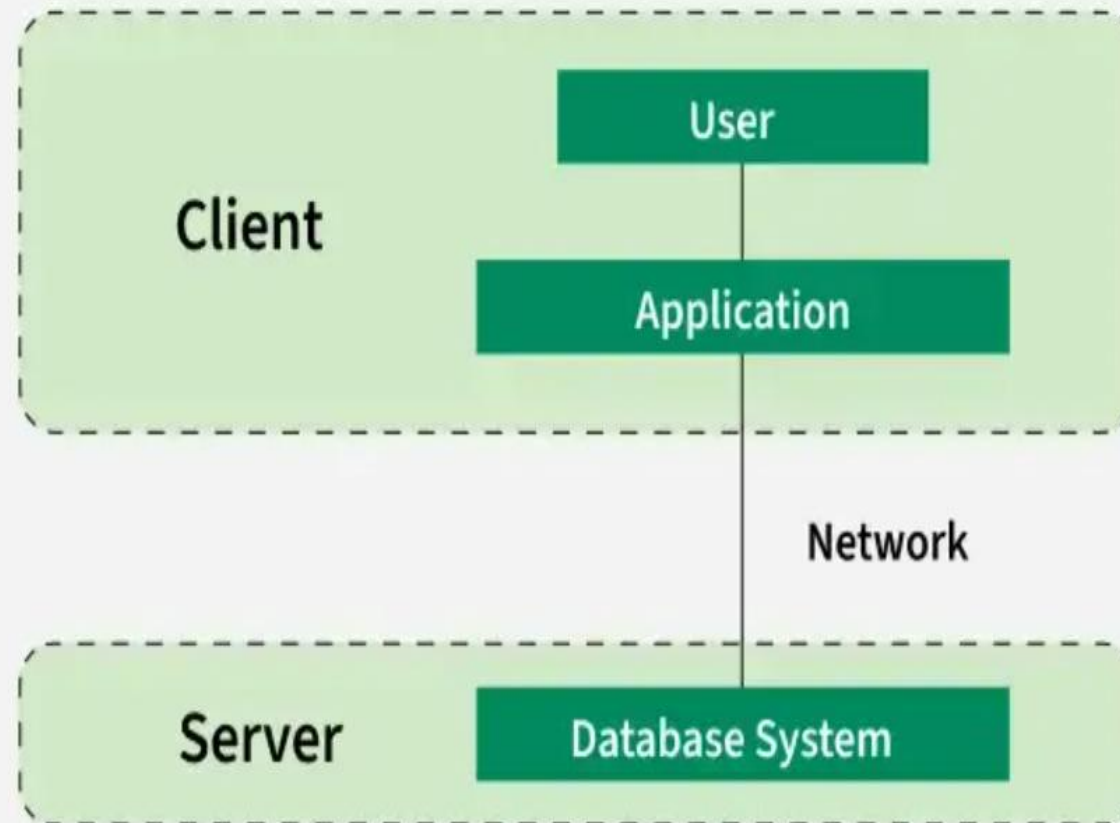
2-Tier Architecture

- The 2-tier architecture is similar to a **basic client-server model**.
- The **application at the client end directly communicates with the database on the server side**.
 - **APIs like ODBC and JDBC are used for this interaction.**
- The **server side is responsible for providing query processing and transaction management functionalities**.

Example

- A **Library Management System** used in schools or small organisations is a classic example of two-tier architecture.
- **Client Layer (Tier 1):** This is the user interface that library staff or users interact with. For example, they might use a desktop application to search for books, issue them, or check due dates.
- **Database Layer (Tier 2):** The database server stores all the library records such as book details, user information and transaction logs.
- The client layer sends a request (like searching for a book) to the database layer, which processes it and sends back the result. This separation allows the client to focus on the user interface, while the server handles data storage and retrieval.

Two-Tier Architecture



2-tier architecture

Advantages of 2-Tier Architecture

- **Easy to Access:** 2-Tier Architecture makes easy access to the database, which makes fast retrieval.
- **Scalable:** We can scale the database easily, by adding clients or upgrading hardware.
- **Low Cost:** 2-Tier Architecture is cheaper than 3-Tier Architecture and Multi-Tier Architecture.
- **Easy Deployment:** 2-Tier Architecture is easier to deploy than 3-Tier Architecture.
- **Simple:** 2-Tier Architecture is easily understandable as well as simple because of only two components.

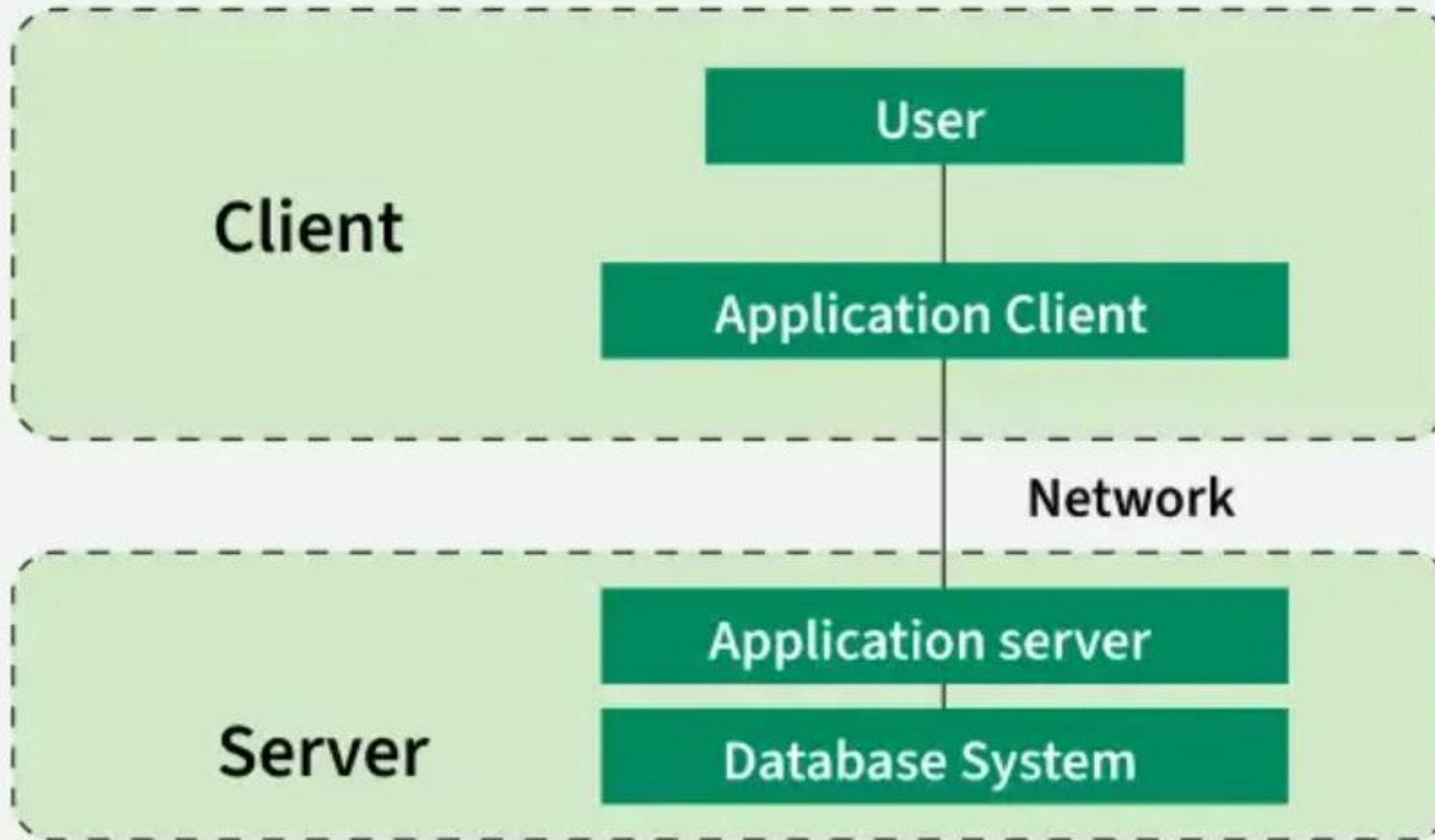
Disadvantages of 2-Tier Architecture

- **Limited Scalability:** As the number of users increases, the system performance can slow down because the server gets overloaded with too many requests.
- **Security Issues:** Clients connect directly to the database, which can make the system more vulnerable to attacks or data leaks.
- **Tight Coupling:** The client and the server are closely linked. If the database changes, the client application often needs to be updated too.
- **Difficult Maintenance:** Managing updates, fixing bugs, or adding features becomes harder when the number of users or systems increases.

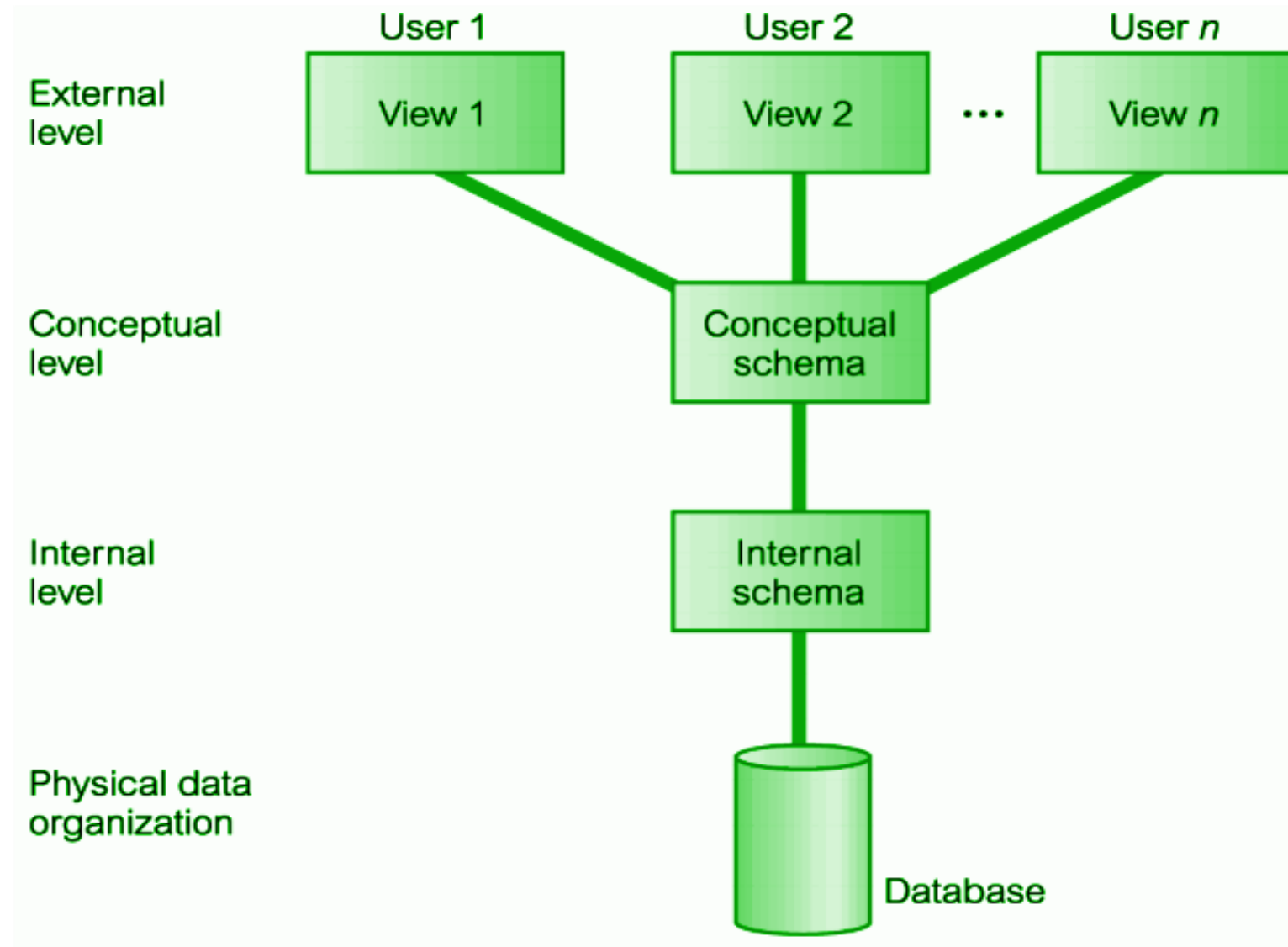
3-Tier Architecture

- In 3-Tier Architecture, **there is another layer between the client and the server.**
- **The client does not directly communicate with the server.** Instead, it interacts with an application server which further communicates with the database system and then the query processing and transaction management takes place.
- **This intermediate layer acts as a medium for the exchange of partially processed data** between the server and the client.
- This type of architecture is used in the case of **large web applications.**

Three-Tier Architecture



3-tier architecture



3-Tier Architecture View

Components of 3-Tier Architecture

1. Presentation Tier (User Interface Layer)

- This tier **interacts directly with users**. It displays data and accepts user input, forwarding requests to the application tier.
- **Examples:** Web browsers, mobile apps, desktop applications

Importance: Enhances user experience by hiding business logic and database complexity.

2. Application Tier (Business Logic Layer)

- This **middle layer processes user requests and implements business rules**. It communicates with both the presentation tier and the data management tier.
- **Examples:** Stock verification, discount calculation, payment processing

Importance: Separates business logic from UI, making the system easier to modify and manage.

Components of 3-Tier Architecture

3. Data Management Tier (Database Layer)

- This tier stores, retrieves, and manages data. It ensures data integrity, security, and efficient access.
- **Examples:** Customer records, transaction data

Importance: Maintains data consistency and allows database changes without affecting application logic.

Key Benefit

- The separation of concerns in 3-tier architecture enables **better scalability, easier maintenance, enhanced security, and independent development of each layer.**

Example: **E-commerce Store**

- **User:** You visit an online store, search for a product and add it to your cart.
- **Processing:** The system checks if the product is in stock, calculates the total price and applies any discounts.
- **Database:** The product details, your cart and order history are stored in the database for future reference.

Advantages of 3-Tier Architecture

- **Enhanced scalability:** Scalability is enhanced due to the distributed deployment of application servers. Now, individual connections need not be made between the client and server.
- **Data Integrity:** 3-Tier Architecture maintains Data Integrity. Since there is a middle layer between the client and the server, data corruption can be avoided/removed.
- **Security:** 3-Tier Architecture Improves Security. This type of model prevents direct interaction of the client with the server thereby reducing access to unauthorized data.

Data Abstraction in DBMS

- Database systems are made-up of complex data structures. To ease the user interaction with database, the developers hide internal irrelevant details from users. This process of hiding irrelevant details from user is called data abstraction.
- **We have three levels of abstraction:**

Physical level: This is the lowest level of data abstraction. It describes how data is actually stored in database. You can get the complex data structure details at this level.

- **Logical level:** This is the middle level of 3-level data abstraction architecture. It describes what data is stored in database.
- **View level:** Highest level of data abstraction. This level describes the user interaction with database system.

- **Example:**
- Let's say we are storing customer information in a customer table. At **physical level** these records can be described as blocks of storage (bytes, gigabytes, terabytes etc.) in memory. These details are often hidden from the programmers.
- At the **logical level** these records can be described as fields and attributes along with their data types, their relationship among each other can be logically implemented. The programmers generally work at this level because they are aware of such things about database systems.
- At **view level**, user just interact with system with the help of GUI and enter the details at the screen, they are not aware of how the data is stored and what data is stored; such details are hidden from them.